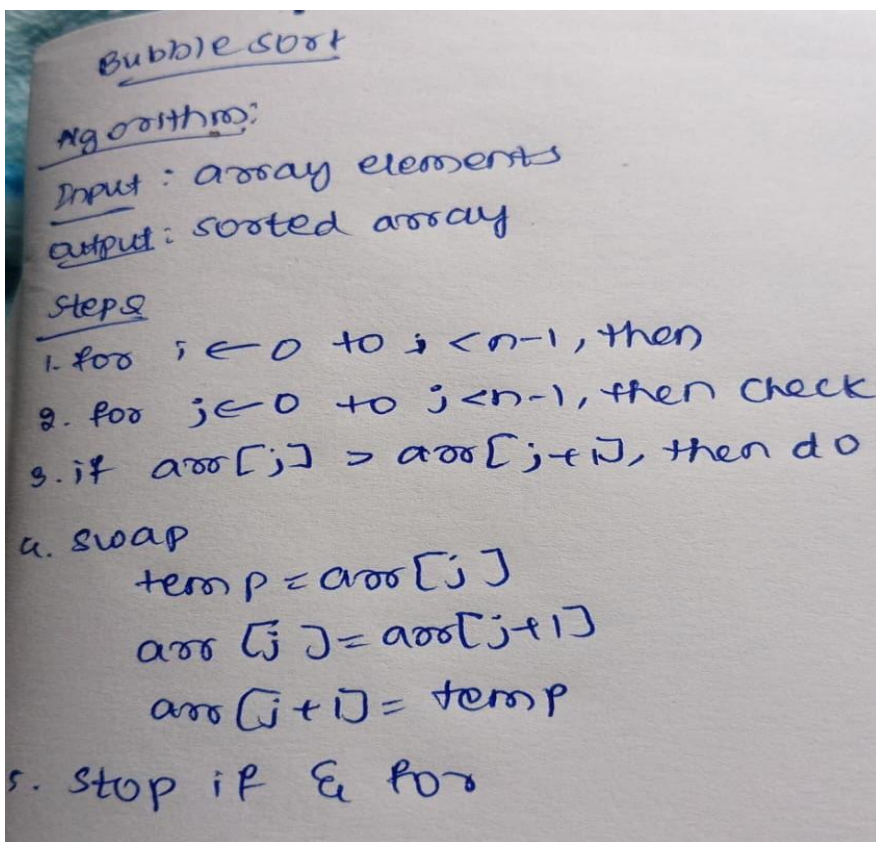


PRACTICAL-1

BUBBLE SORT

AIM: Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

ALGORITHM:



CODE:

```
#include <iostream>

using namespace std; int

main() {

    int n;
```

```
// Input number of elements    cout <<
"Enter the number of elements: ";    cin >>
n;

int arr[n];

// Input array elements    cout <<
"Enter " << n << " elements: ";    for (int
i = 0; i < n; i++) {    cin >> arr[i];
}

// Bubble Sort    for (int i =
0; i < n - 1; i++) {

    for (int j = 0; j < n - i - 1; j++) {

        // Compare adjacent elements
if (arr[j] > arr[j + 1]) {

            // Swap elements

            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

```
}  
  
// Display sorted array  
cout << "Sorted array: ";  
  
for (int i = 0; i < n; i++) {  
cout << arr[i] << " ";  
}  
  
return 0;  
}
```

OUTPUT:

```
Output  
  
Enter the number of elements: 4  
Enter 4 elements: 5  
4  
7  
8  
Sorted array: 4 5 7 8  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:

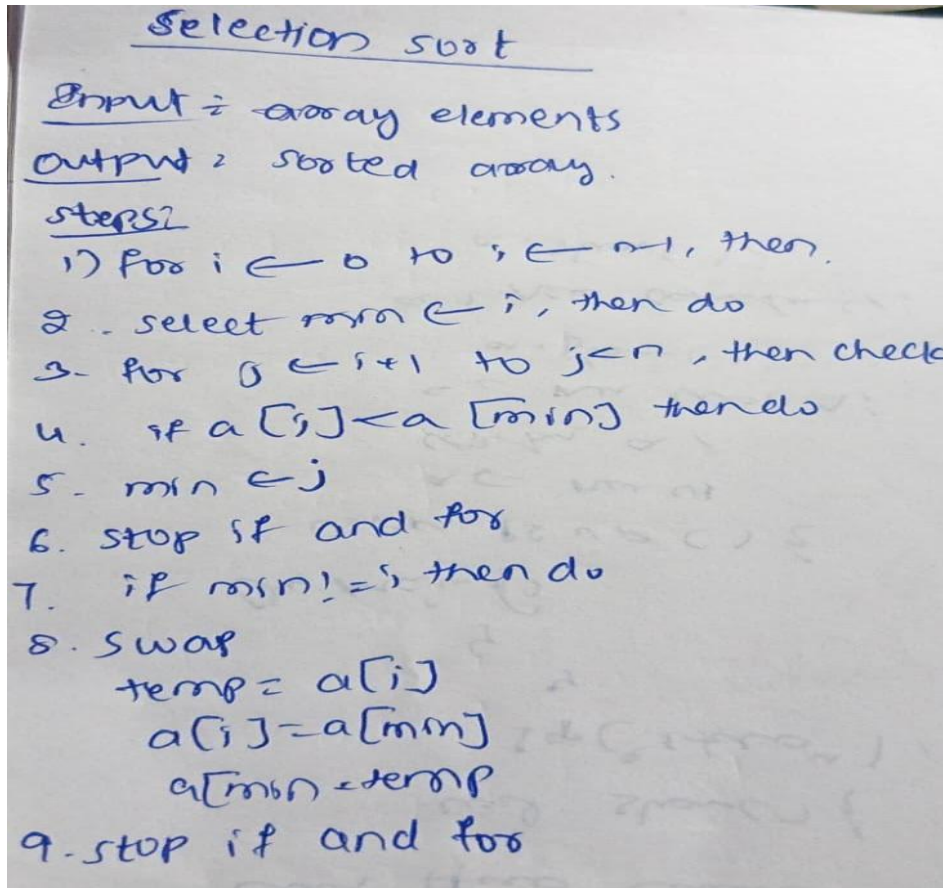
Time Complexity: (Avg & Worst Cases):
 for (int i = 0; i < n-1; i++) { $\rightarrow n-1$
 for (int j = 0; j < n-1; j++) { $\rightarrow n-1$
 if (arr[j] > arr[j+1]) { $\rightarrow 1$
 temp = arr[j]; $\rightarrow 1$
 arr[j] = arr[j+1]; $\rightarrow 1$
 arr[j+1] = temp; $\rightarrow 1$
 }
 }
 $T(n) = (n-1)(n-1) \times 1 \times 1 \times 1 \times 1$
 $= n^2 - n \times 1 = n^2 - n$
 $T(n) = O(n^2)$

BEST CASE:

Best Case: when the array is
 already sorted.
 $\rightarrow$ This array will be sorted for
 first pass only
 $T(n)$ for one pass = $n-1$
 So $T(n) = n-1$
 $\therefore T(n) = O(n)$

SELECTION SORT:

ALGORITHM:



CODE:

```
#include <iostream> using
```

```
namespace std;
```

```
class SelectionSort
```

```
{ public:
```

```
    int size, arr[100], i, j, temp;
```

```
    void arrayinput()
```

```
{
```

```
cout << "Enter array size: ";  
cin >> size;  
  
cout << "Enter array elements: ";  
  
for (i = 0; i < size; i++)  
{  
    cin >>  
arr[i];  
}  
  
void printarray()  
{  
    for (i = 0; i < size;  
i++)  
    {  
        cout << arr[i] << " ";  
    }  
  
    cout << endl;  
}  
  
void selectionsortlogic()  
{  
    for (i = 0; i < size - 1;  
i++)
```

```
{      int
min = i;
for (j = i + 1; j <
size; j++)
{
    if (arr[j] < arr[min])
    {
min = j;
    }
}
    if (min !=
i)
    {
        temp =
arr[i];        arr[i] =
arr[min];
arr[min] = temp;
    }
}
};

int main()
{
    SelectionSort ss;
```

```
ss.arrayinput();

cout << "Before sorting: ";  ss.printarray();

ss.selectionsortlogic();

cout << "After sorting: ";
ss.printarray();

return 0;
}
```

OUTPUT:

```
Output

Enter array size: 4
Enter array elements: 7
8
25
78
Before sorting: 7 8 25 78
After sorting: 7 8 25 78

=== Code Execution Successful
```

TIME COMPLEXITY:

Time complexity : (Best ~~avg~~ avg, worst cases) :

```

for (i = 0; i < n-1; i++) { → n-1
    int min = i;
    for (j = i+1; j < n; j++) → n
    {
        if (arr[j] < arr[min]) { → 1
            min = j; → 1
        }
    }
    if (min != i) { → 1
        int temp = arr[i]; → 1
        arr[i] = arr[min]; → 1
        arr[min] = temp; → 1
    }
}

```

$T(n) = (n-1)(n) \times 1 \times 1 \times 1 \times 1 \times 1$
 $= n^2 - n$
 ~~$T(n) = O(n^2)$~~
 $T(n) = O(n^2)$

In Selection Sort:
 Best = Average = worst = $O(n^2)$

INSERTION SORT

ALGORITHM:

insertion sort

Input: Array elements
Output: sorted array
steps:

- 1) for $i \leftarrow 1$ to n , then
- 2) $temp \leftarrow a[i]$ and $j \leftarrow i-1$ then
- 3) while $j \geq 0$ and $a[j] > temp$
- 4) $a[j+1] \leftarrow a[j]$ and $j--$
- 5) stop while.
- 6) $a[j+1] \leftarrow temp$ and
- 7) stop for

CODE:

```
#include <iostream> using
namespace std;

int main()
{   int
n;

    cout << "Enter the number of elements: ";
cin >> n;

    int arr[n];

    cout << "Enter " << n << " elements: ";

    for (int i = 0; i < n; i++)
    {   cin >>
arr[i];
    }

    // Insertion Sort    for
(int i = 1; i < n; i++)
    {   int key =
arr[i];    int j = i -
```

```
1;    while (j >= 0
&& arr[j] > key)

    {        arr[j + 1] =
arr[j];
        j--;
    }

    arr[j + 1] = key;
}

cout << "Sorted array: ";

for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
}

return 0;
}
```

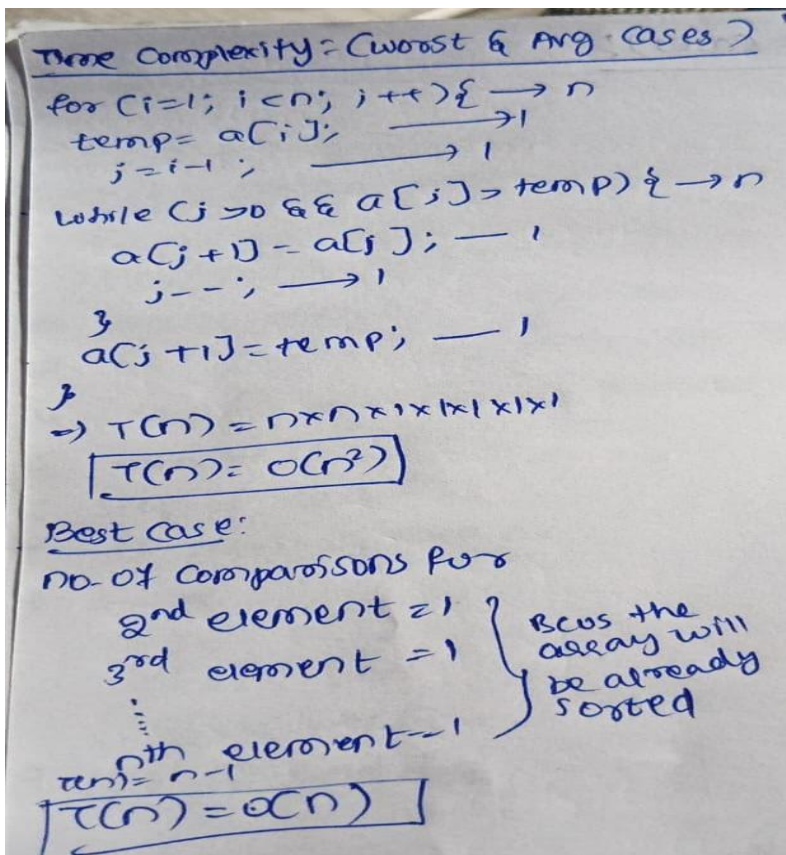
OUTPUT:

Output

```
Enter the number of elements: 4
Enter 4 elements: 4
5
35
45
Sorted array: 4 5 35 45
```

=== Code Execution Successful ===

TIME COMPLEXITY:



MERGE SORT

ALGORITHM:

merge sort:
Input: Array elements
Output: Sorted Array
Steps:
1. merge (arr, lb, mid, ub) then
2. set $i = lb, j = mid + 1, k = lb$
3. while $i \leq mid$ and $j \leq ub$, then
4. if $arr[i] < arr[j]$, then
 $temp[k] = arr[i]$
 $i = i + 1$
 else
 $temp[k] = arr[j]$
 $j = j + 1$
 end if
 $k = k + 1$
 end while
5. while $j \leq ub$ then
 $temp[k] = arr[j]$
 $j = j + 1$
 $k = k + 1$
 end while
6. while $i \leq mid$ then
 $temp[k] = arr[i]$
 $i = i + 1$
 $k = k + 1$
 end while
7. for $i = lb, k = 0$ to $i \leq ub$
 then $arr[i] = temp[k]$
8. stop for

CODE:

```
#include <iostream> using
namespace std;

void merge(int arr[], int low, int mid, int high)
{   int i = low;
    int j = mid + 1;
    int k = 0;

    int temp[high - low + 1];

    while (i <= mid && j <= high)
    {   if (arr[i] <
arr[j])
        {
            temp[k] = arr[i];
i++;   }   else
        {
            temp[k] = arr[j];
j++;
        }

        k++;
    }
}
```

```
while (i <= mid)
{
    temp[k] = arr[i];
    i++;    k++;
}

while (j <= high)
{
    temp[k] = arr[j];
    j++;    k++;
}

for (int i = low, k = 0; i <= high; i++, k++)
{
    arr[i] = temp[k];
}
}

void mergeSort(int arr[], int low, int high)
{
    if (low <
high)
    {
        int mid = (low + high) / 2;

        mergeSort(arr, low, mid);
```

```
mergeSort(arr, mid + 1, high);

merge(arr, low, mid, high);
}
}

int main()
{   int
n;

cout << "Enter number of elements: ";
cin >> n;

int arr[n];

cout << "Enter " << n << " elements: ";

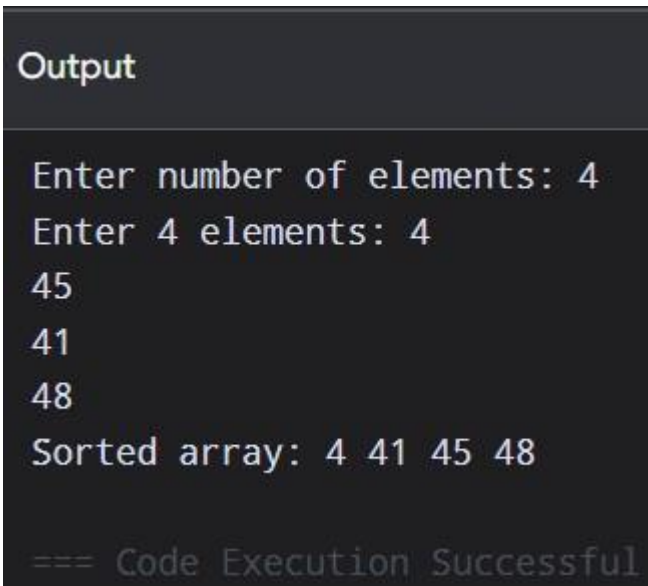
for (int i = 0; i < n; i++)
{   cin >>
arr[i];
}

mergeSort(arr, 0, n - 1);

cout << "Sorted array: ";
```

```
for (int i = 0; i < n; i++)  
{  
    cout << arr[i] << " ";  
}  
  
return 0;  
}
```

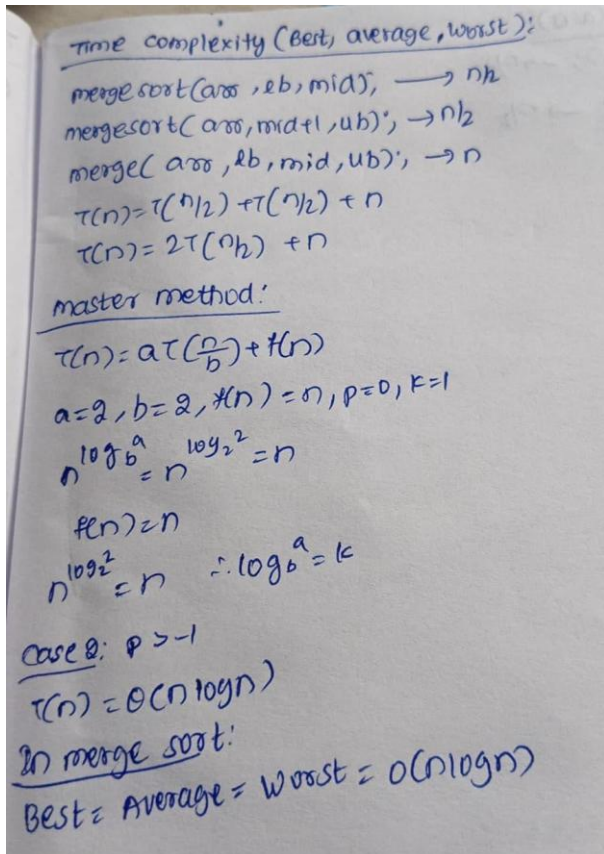
OUTPUT:



The screenshot shows a terminal window with a dark background. The title bar reads 'Output'. The text in the terminal is as follows:

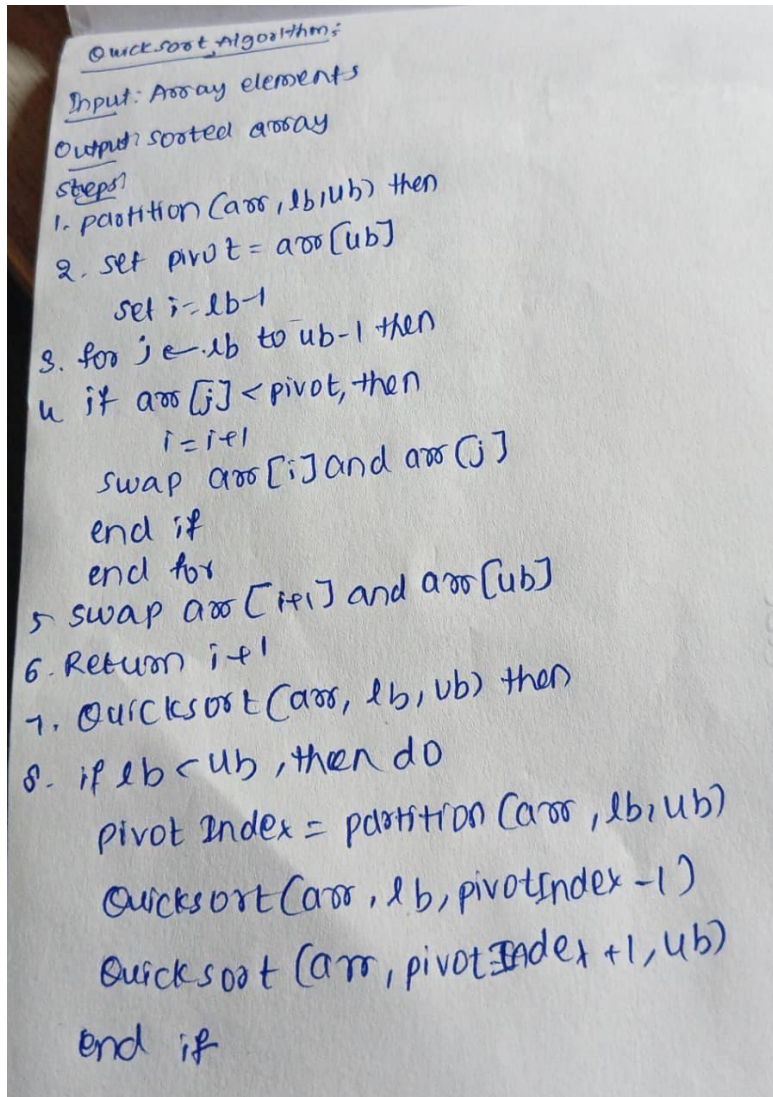
```
Enter number of elements: 4  
Enter 4 elements: 4  
45  
41  
48  
Sorted array: 4 41 45 48  
  
=== Code Execution Successful
```

TIME COMPLEXITY:



QUICK SORT

ALGORITHM:



CODE:

```
#include <iostream> using
```

```
namespace std;
```

```
int partition(int arr[], int low, int high)
```

```
{   int pivot =
```

```
arr[high];   int i = low
```

```
- 1;
```

```
for (int j = low; j < high; j++)
{
    if (arr[j] <
pivot)
    {
        i++;

        int temp = arr[i];
arr[i] = arr[j];        arr[j]
= temp;
    }
}

int temp = arr[i + 1];
arr[i + 1] = arr[high];
arr[high] = temp;

return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low <
high)
    {
        int pi = partition(arr, low, high);        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
    }
}

int main()
{    int
n;

    cout << "Enter number of elements: ";
cin >> n;

    int arr[n];

    cout << "Enter " << n << " elements: ";

    for (int i = 0; i < n; i++)
    {        cin >>
arr[i];
    }

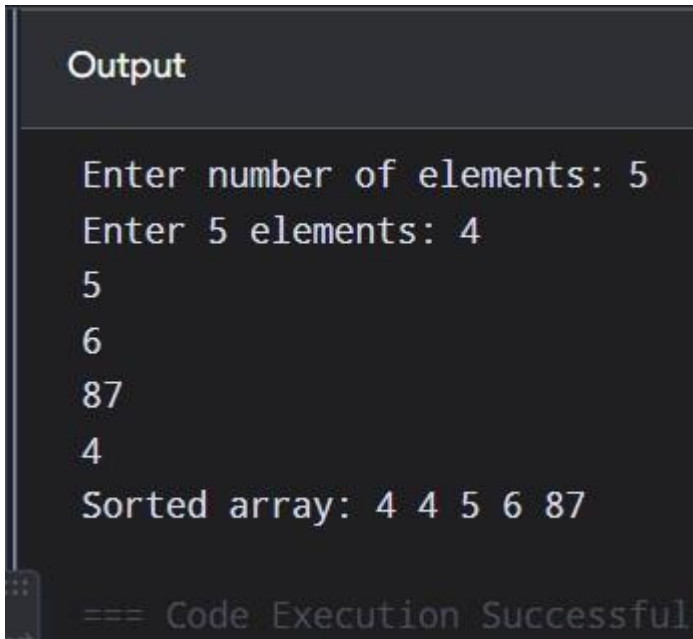
    quickSort(arr, 0, n - 1);

    cout << "Sorted array: ";

    for (int i = 0; i < n; i++)
```

```
{  
    cout << arr[i] << " ";  
}  
  
return 0;  
}
```

OUTPUT:



```
Output  
  
Enter number of elements: 5  
Enter 5 elements: 4  
5  
6  
87  
4  
Sorted array: 4 4 5 6 87  
  
=== Code Execution Successful
```

TIME COMPLEXITY:

Time complexity: (Best & Average cases):
 $\text{quicksort}(\text{arr}, \text{lb}, \text{pivotIndex}-1); \rightarrow n/2$
 $\text{quicksort}(\text{arr}, \text{pivotIndex}+1, \text{ub}); \rightarrow n/2$
 $\text{partition}(\text{arr}, \text{lb}, \text{ub}); \rightarrow n$
 $T(n) = T(n/2) + T(n/2) + n$
 $T(n) = 2T(n/2) + n$

Master method:
 $T(n) = 2T(n/2) + n$
 $a = 2, b = 2, f(n) = n^2, p = 0, k = 1$
 $n^{\log_b a} = n^{\log_2 2} = n$
 $\therefore \log_b a = k$

case 2: $p = -1$
 $T(n) = O(n^{\log_b a + 1})$
 $T(n) = O(n \log n)$
 for Best & Avg case

worst case:
 $\begin{matrix} \text{1} \\ \downarrow \\ n-1 \\ \downarrow \\ n-2 \\ \downarrow \\ n-3 \\ \downarrow \\ n-4 \\ \vdots \end{matrix} \Rightarrow \begin{matrix} T(n) = T(n-1) + n \\ T(n) = \frac{n(n+1)}{2} \\ T(n) = n^2 + n \\ T(n) = O(n^2) \end{matrix}$
 for worst case.

CONCLUSION:

The practical successfully implemented and analyzed five important sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. By comparing their working principles and time complexities, it was observed that Bubble Sort, Selection Sort, and Insertion Sort generally take $O(n^2)$ time in the average and worst cases, while Merge Sort provides $O(n \log n)$ performance consistently. Quick Sort also has an average time complexity of $O(n \log n)$, although its worst case can be $O(n^2)$ depending on the choice of pivot.

Thus, the practical helped in understanding how different sorting techniques work, how their performance varies with input size, and how time complexity is useful for selecting an efficient algorithm for a particular problem.